bhttp/1: HTTP in binary frames

Version 1 · Raj Prakash · A request/response protocol with HTTP semantics, carried in length-prefixed binary frames over one persistent TCP connection.

1. Conventions

The key words MUST, MUST NOT, SHOULD, SHOULD NOT and MAY are used as in RFC 2119. All integers are unsigned and **big-endian** (network byte order); widths are in bits. A *message* is a request or a response. Hex bytes are written `0x2f` or `2f`.

2. Connection

bhttp/1 runs over TCP. There is no registered port; examples use 9000. There is no preface or handshake: the client's first bytes are its first frame.

- The connection is **persistent** and **sequential**: it carries any number of requests, one at a time. A client MUST NOT send a REQUEST before the previous response is complete (its END has arrived). A server MUST answer requests in the order received.
- The **client closes** when it has no more requests; this is the normal end. It MAY send GOAWAY first.
- The **server closes** (a) after a connection error (§7), or (b) when idle: T seconds pass without a complete REQUEST (inside a request body: without a complete DATA frame of it); other frames, such as unknown types, and partial frames do not extend T (T is implementation-defined, SHOULD be at least 10; bserve uses 30). Before closing on its own initiative a server SHOULD send GOAWAY, then SHOULD shut down its sending side and read and discard input for a short time (≤ 2 s) before closing, so that the peer receives the GOAWAY rather than a reset.
- A server MAY refuse a connection it cannot take (bserve: beyond 64 at once) by sending GOAWAY with Last-ID 0 and Code `0x02` BUSY, then closing; the client MAY retry later.
- A receiver whose connection ends in the middle of a frame discards the partial frame and closes. Nothing is sent in reply.

3. Frame header

Every frame is an 8-byte header followed by `Length` bytes of payload.

```
byte  0     1     2     3     4     5     6     7
    +-----+-----+-----+-----+-----+-----+-----+-----+
    |  Length   |Type |Flags|      Request ID       |
    +-----+-----+-----+-----+-----+-----+-----+-----+
         16        8     8            32
```

- **Length (16):** payload size in bytes, excluding the header (an empty frame is 8 bytes). MUST be ≤ **16384** (`MAX_PAYLOAD`). A larger value is a connection error (§7) whatever the Type, so a receiver never needs a buffer larger than 8 + 16384 bytes.
- **Type (8):** the frame type (§4).
- **Flags (8):** `0x01` = **END**. Other bits are unused in v1: senders MUST set them to 0; receivers MUST ignore them.
- **Request ID (32):** which request the frame belongs to. The client numbers its requests 1, 2, 3, … on each connection (MUST be nonzero, SHOULD increase by one). The server copies the ID into every frame of the response and checks only that it is not 0. ID 0 means the connection itself: a REQUEST, RESPONSE or DATA with ID 0 sent to a server is a connection error (§7), and a RESPONSE with ID 0 reports one.

Why these widths. HTTP/2 uses Length 24, Type 8, Flags 8, 1 reserved bit + Stream ID 31 (9 bytes). bhttp/1 uses 16 / 8 / 8 / 32 (8 bytes):

Field	HTTP/2	bhttp	Reason
Length	24	16	v1 frames carry at most 16 KiB (HTTP/2's default limit too): 15 bits; the 16th lets a later version negotiate frames up to 64 KiB. HTTP/2 needs 24 only because SETTINGS can raise its limit to 16 MiB; at 16 KiB the header is already 0.05 % overhead. Bonus: text such as `GET /` starts with a letter ≥ `0x41`, reads as Length ≥ 16640, and is rejected on its first two bytes.
Type	8	8	Same as HTTP/2. v1 uses 4 values; the other 252 are the room for version 2 (§9).
Flags	8	8	Same. v1 needs one bit (END); seven spare bits cost nothing.
ID	1+31	32	v1 does not multiplex, so order alone could match responses. The ID is kept because (1) a client detects a desynchronised connection instead of printing the wrong page, and (2) the header is the one thing no later version may change (every receiver must find Length in the same place to skip unknown frames), so room for multiplexing is reserved now. HTTP/2's reserved bit is a SPDY leftover; 32 full bits never wrap and keep the header at 8 bytes, every field naturally aligned.

4. Frame types

Type	Name	Sent by	Request ID	Payload
`0x01`	REQUEST	client	the new request's ID	Method, Path, Header block (§5.1)
`0x02`	RESPONSE	server	copied from the request	Status, Header block (§5.2)
`0x03`	DATA	both	the message's ID	0–16384 bytes of body
`0x04`	GOAWAY	both	0	Last-ID (32), Code (8)

END on REQUEST or RESPONSE means "no body follows"; on DATA, "last frame of the body"; on GOAWAY it is ignored.

Unknown types, the MUST-skip rule. A receiver that meets a frame whose Type it does not know MUST read and discard exactly `Length` payload bytes and carry on as if the frame were absent, whatever its Flags and ID and wherever it appears (also between the frames of a message). It MUST NOT reply to it or close because of it; only the Length limit applies. Types `0x00` and `0x05–0x7f` are reserved for later versions of this spec, `0x80–0xff` for private experiments. A v1 sender MUST NOT send any of them.

GOAWAY says "I will close; send no more requests". Its Request ID is 0 and is ignored on receipt. *Last-ID* is, from a server, the ID of the last request it answered completely (0 if none); a client sends 0. *Code*: `0x00` NO_ERROR (idle timeout, shutdown), `0x01` PROTOCOL_ERROR, `0x02` BUSY (§2); any other value is treated as `0x01`. Bytes after the first five are ignored; a shorter payload is still a GOAWAY, read as Last-ID 0 and Code `0x01`. The receiver MUST NOT send another REQUEST and SHOULD close. Requests with an ID above Last-ID were not processed and MAY be retried on a new connection.

5. Payloads

5.1 REQUEST: `Method (8) | Path Length (16) | Path | Header block`. Methods: `0x01` GET, `0x02` HEAD, `0x03` POST, `0x04` PUT, `0x05` DELETE, with HTTP meaning; any other code is malformed. The Path is 1 to `Length` – 3 raw bytes (nobody percent-decodes: `%2e%2e` is a literal name). Everything from the first `?` is the *query*. The Path is malformed if it contains a byte `0x00–0x1f` or `0x7f`, or if the part before the query does not start with `/` or has a `/`-separated segment equal to `.` or `..`. The header block fills the rest of the payload and may be empty.

5.2 RESPONSE: `Status (16) | Header block`. Status is an HTTP status code from 200 to 599 (v1 has no 1xx responses). bserve sends 200, 301, 400, 403, 404, 405 and 500.

5.3 Header block. Zero or more entries back to back, ending exactly at the end of the payload. An entry starts with an *Index* byte that selects one of two forms:

```
indexed:  | Index 1..10 | VLen (16) | Value |
literal:  | 0x00 | NLen (8) | Name | VLen (16) | Value |
```

Index is 8 bits, NLen (name length) 8 bits, VLen (value length) 16 bits. The static table:

Index	Name	Index	Name
1	`host`	6	`last-modified`
2	`user-agent`	7	`date`
3	`accept`	8	`server`
4	`content-type`	9	`location`
5	`content-length`	10	`allow`

These are exactly the ten names bcurl and bserve send: 1–3 by the client, 4–10 by the server (9 with a 301, 10 with a 405). A name in the table SHOULD be sent indexed; receivers MUST accept the literal form too. An entry is malformed if its Index is 11–255, if it runs past the end of the payload, if a literal Name is empty or has a byte other than lowercase `a-z 0-9 ! # $ % & ' * + - . ^ _ | ~` and backtick, or if a Value contains `0x00`, `0x0a` or `0x0d`. Order is kept and repeated names are allowed, with HTTP meaning; `date` and `last-modified` use IMF-fixdate (`Thu, 08 Oct 2026 12:08:46 GMT`). Clients SHOULD send `host`; a server MUST NOT require any header. There is no dynamic table and no Huffman coding, and the block must fit in its one frame.

6. Messages and bodies

A message is a REQUEST or RESPONSE frame and, if that frame lacks END, a body of DATA frames with the same ID, the last one carrying END. (A server receives only REQUESTs, so only a REQUEST opens a body there.) Each DATA frame carries ≤ 16384 bytes; senders SHOULD fill all but the last. An empty DATA frame is allowed (useful only with END).

`content-length` is optional. If sent, its value is 1 to 19 ASCII digits (leading zeros allowed), any further copy MUST be byte-for-byte identical (`3` and `03` are not), and it MUST equal the body length. The one exception is HEAD: every response to a REQUEST whose Method byte is HEAD, whatever its status (a 400 too), sets END on the RESPONSE and has no DATA; a 200 carries the headers a GET would get. A receiver treats a violation as a malformed message.

A server MUST read a request body through its END before it treats any further frame as a new request; bserve reads the body first and then responds. A server that fails after sending RESPONSE (for example, a read error mid-file) MUST close the connection without sending END.

7. Errors

Connection errors break the framing; the stream cannot be resynchronised. They are a Length above 16384 (the frame is not read further); a REQUEST, RESPONSE or DATA with ID 0; and, while a request body is open, any frame other than DATA for that ID, an unknown type, or GOAWAY (which simply ends the connection). The server sends RESPONSE **400** with ID 0 (with, like any response, an optional body under that ID), then GOAWAY (`0x01`), then closes as in §2.

Request errors leave the framing intact: the next frame starts where expected. The server replies RESPONSE **400** with the offending frame's ID and **keeps the connection open**. They are a REQUEST payload under 4 bytes, an unknown method, a Path Length of 0 or past the payload, a Path rule violation (§5.1), a malformed header block, a bad `content-length` (§6), a RESPONSE frame sent to the server (it opens no body), and a DATA frame with no open request body. A malformed REQUEST without END still has its body read first (§6).

Other replies: 404 not found, outside the root, hidden, or not a regular file; **403** not readable; **301** with `location` = path (without the query) + `/` for a directory requested without a trailing `/`; **405** with `allow` for a method the server does not implement (bserve: `GET, HEAD`); **500** for a read error before RESPONSE. Error responses MAY carry a short `text/plain` body.

Clients: a frame with Length > 16384, a RESPONSE or DATA with a wrong ID (including a RESPONSE with ID 0, which reports a connection error), a DATA before the RESPONSE, a second RESPONSE, a REQUEST, a malformed payload, a status outside 200–599, a bad `content-length`, or EOF before END is a protocol error: close and report failure. A GOAWAY before END means the request was not processed (its ID is above Last-ID): close; it MAY be retried on a new connection.

8. Path mapping

A server serving files from a directory ROOT maps the Path as follows: drop the query; if the path ends in `/`, append `index.html`; if any segment starts with `.`, 404; resolve ROOT + path with all symbolic links followed, and if the result is not inside ROOT (resolved the same way), 404; a directory gets 301 (§7), or 404 if the path already ended in `/`; anything else that is not a regular file, 404. This MUST hold for what is actually opened even if the tree changes meanwhile: bserve opens the resolved name component by component from ROOT without following links, so a link swapped in gives 404. Syntax errors are 400 (§5.1), so `..`, a Path without the leading `/`, and NUL bytes never reach the file system. `content-type` SHOULD follow the file extension. bserve: html, htm → text/html; css → text/css; js → text/javascript; txt → text/plain; md → text/markdown (these five with `; charset=utf-8`); json → application/json; pdf → application/pdf; png → image/png; gif → image/gif; jpg, jpeg → image/jpeg; svg → image/svg+xml; ico → image/x-icon; any other → application/octet-stream.

9. Versions

There is no version number on the wire. The frame header and the MUST-skip rule are frozen for every version. A later version may add frame types (from `0x05-0x7f`), flag bits, GOAWAY codes or trailing GOAWAY bytes, and frames up to Length 65535, but uses them only after the peer has shown it understands them: a v2 peer announces itself with a new frame type (say, HELLO) that a v1 peer silently skips. A v1 server never sends such a frame, so a v2 client that gets a RESPONSE without the server's announcement stays at v1.

10. Worked example

GET `/` with no headers, ID 1, and a 404 answer with no body:

```
00 04 01 01 00 00 00 01   Length 4, REQUEST, END, ID 1
01 00 01 2f               GET, Path Length 1, "/"
00 02 02 01 00 00 00 01   Length 2, RESPONSE, END, ID 1
01 94                     Status 0x0194 = 404
```

Header entries: `01 00 0e` + `localhost:9000` is `host: localhost:9000`, and `00 03 78 2d 61 00 01 31` is the literal `x-a: 1`. HEXDUMP.md annotates a full exchange.

One connection, frame by frame (C = client, S = server):

```
C>S  REQUEST  id=1 END   GET /a.bin
S>C  RESPONSE id=1       200, content-length: 20000
S>C  DATA     id=1       16384 bytes
S>C  DATA     id=1 END   3616 bytes
C>S  type 0x42 id=0      unknown: skipped, no reply
C>S  REQUEST  id=2 END   GET /x/../y  (dot segment)
S>C  RESPONSE id=2       400  (request error: kept open)
S>C  DATA     id=2 END   "400 Bad Request: ...\n"
       ... 30 s with no frame from the client ...
S>C  GOAWAY   id=0       Last-ID 2, NO_ERROR; S closes
```

11. Reference client

Informative: the contract of bcurl. `bcurl [-v] URL...`, where URL is `[bhttp://]host[:port][path]`. The port defaults to 9000; the host ends at the first `:`, `/`, `?` or `#`; a `#fragment` is never sent; an empty path becomes `/` and a path that starts with `?` gets a `/` in front; otherwise the path is sent unchanged. All URLs must name the same host and port and are fetched in order over one connection, never reopened. Exit status: 0 if every response was 2xx or 3xx, 4 if the worst was 4xx, 5 if 5xx, 2 on a connection or protocol error (later URLs are not fetched), 1 on a usage error, which includes a URL whose REQUEST would not fit in one frame (nothing is sent); the worst wins, in the order 2, 5, 4, 0.

12. Conformance checklist

- [ ] Both: big-endian; Length > 16384 is a connection error; unknown types skipped by exactly Length bytes; unknown flags ignored; reserved types never sent; ten-name table and literal form; DATA reassembled through END; `content-length` checked.
- [ ] Client: IDs 1, 2, 3, … on one connection; one request at a time; the ID of every RESPONSE and DATA checked; protocol errors (§7) end the connection.
- [ ] Server: ID copied to RESPONSE and DATA; request errors get 400 and keep the connection; connection errors get 400 (ID 0), GOAWAY `0x01` and a close; request bodies read through END; path rules (§5.1, §8); idle close with GOAWAY `0x00`.